

SISTEMAS DISTRIBUÍDOS

princípios e paradigmas

2ª edição



Andrew S. Tanenbaum
Maarten Van Steen

PEARSON
Prentice
Hall



SISTEMAS DISTRIBUÍDOS

princípios e paradigmas

2ª edição

**Andrew S. Tanenbaum
Maarten Van Steen**

Tradução

Arlene Simille Marques
Engenheira Química — UFPR

Revisão Técnica

Wagner Luiz Zucchi
Professor doutor do Departamento de Sistemas Eletrônicos da
Escola Politécnica da Universidade de São Paulo



São Paulo



Brasil Argentina Colômbia Costa Rica Chile Espanha Guatemala México Peru Porto Rico Venezuela

Sumário

Prefácio IX

1. Introdução 1

1.1 Definição de um sistema distribuído	1
1.2 Metas	2
1.3 Tipos de sistemas distribuídos	10
1.4 Resumo	18

2. Arquiteturas 20

2.1 Estilos arquitetônicos	20
2.2 Arquiteturas de sistemas	22
2.3 Arquiteturas <i>versus</i> middleware	32
2.4 Autogerenciamento em sistemas distribuídos	35

3. Processos 42

3.1 Threads	42
3.2 Virtualização	48
3.3 Clientes	50
3.4 Servidores	53
3.5 Migração de código	62
3.6 Resumo	67

4. Comunicação 69

4.1 Fundamentos	69
4.2 Chamada de procedimento remoto	75
4.3 Comunicação orientada a mensagem	84
4.4 Comunicação orientada a fluxo	95
4.6 Resumo	105

5. Nomeação 108

5.1 Nomes, identificadores e endereços	108
5.2 Nomeação simples	110
5.3 Nomeação estruturada	118
5.4 Nomeação baseada em atributo	131
5.5 Resumo	137

6. Sincronização 140

6.1 Sincronização de relógios	140
6.2 Relógios lógicos	147
6.3 Exclusão mútua	152
6.4 Posicionamento global de nós	157
6.5 Algoritmos de eleição	159
6.6 Resumo	163

7. Consistência e replicação 165

7.1 Introdução	165
7.2 Modelos de consistência centrados em dados	167
7.3 Modelos de consistência centrados no cliente	174
7.4 Gerenciamento de réplicas	178
7.5 Protocolos de consistência	185
7.6 Resumo	191

8. Tolerância a falha 194

8.1 Introdução à tolerância a falha	194
8.2 Resiliência de processo	198

viii Sistemas distribuídos

8.3 Comunicação confiável cliente-servidor.....	203
8.4 Comunicação confiável de grupo	207
8.5 Comprometimento distribuído.....	215
8.6 Recuperação.....	220
8.7 Resumo	226

9. Segurança 228

9.1 Introdução à segurança	228
9.2 Canais seguros	240
9.3 Controle de acesso.....	250
9.4 Gerenciamento da segurança	259
9.5 Resumo	266

10. Sistemas distribuídos baseados em objetos 268

10.1 Arquitetura	268
10.2 Processos.....	272
10.3 Comunicação	275
10.4 Nomeação	281
10.5 Sincronização.....	284
10.6 Consistência e replicação.....	285
10.7 Tolerância a falha.....	288
10.8 Segurança.....	291
10.9 Resumo	294

11. Sistemas de arquivos distribuídos 296

11.1 Arquitetura.....	296
11.2 Processos	302
11.3 Comunicação	303
11.4 Nomeação.....	306
11.5 Sincronização	310
11.6 Consistência e replicação	314
11.7 Tolerância a falha	320
11.8 Segurança	322
11.9 Resumo	328

12. Sistemas distribuídos baseados na Web 330

12.1 Arquitetura	330
12.2 Processos.....	335
12.3 Comunicação	339
12.4 Nomeação	344
12.5 Sincronização.....	345
12.6 Consistência e replicação	346
12.7 Tolerância a falha.....	353
12.8 Segurança.....	354
12.9 Resumo	355

13. Sistemas distribuídos baseados em coordenação 357

13.1 Introdução a modelos de coordenação.....	357
13.2 Arquiteturas	358
13.3 Processos	364
13.4 Comunicação	364
13.5 Nomeação.....	365
13.6 Sincronização	367
13.7 Consistência e replicação	367
13.8 Tolerância a falha	371
13.9 Segurança	374
13.10 Resumo	375

14. Sugestões de leitura adicional e bibliografia 377

14.1 Sugestões para leitura adicional.....	377
14.2 Bibliografia em ordem alfabética.....	382

Prefácio

Sistemas distribuídos são uma área da ciência da computação que está mudando rapidamente. Nos últimos anos vêm surgindo novos tópicos muito interessantes, como computação peer-to-peer e redes de sensores, enquanto outros amadureceram muito, como serviços Web e aplicações Web em geral. Mudanças como essas exigiram uma revisão de nosso texto original para que ficasse atualizado.

Esta segunda edição reflete uma grande revisão em comparação com a anterior. Adicionamos um capítulo específico sobre arquiteturas, que reflete o progresso alcançado na organização de sistemas distribuídos. Uma outra diferença importante é que, agora, há muito mais material sobre sistemas descentralizados, em particular computação peer-to-peer. Não nos limitamos a discutir apenas as técnicas básicas, mas também demos atenção às suas aplicações, como compartilhamento de arquivo, divulgação de informações, redes de entrega de conteúdo e sistemas publicar/subscrever.

Assim como esses dois assuntos importantes, ainda discutimos novos assuntos em todo o livro. Por exemplo, adicionamos material sobre redes de sensores, virtualização, clusters de servidores e computação em grade. Demos atenção especial ao autogerenciamento de sistemas distribuídos, um tópico cada vez mais importante dado o contínuo crescimento da escala desses sistemas.

Certamente, modernizamos o material onde foi adequado. Por exemplo, quando discutimos consistência e replicação, era agora focalizamos modelos de consistência mais apropriados para sistemas distribuídos modernos mais do que os modelos originais, que foram talhados para computação distribuída de alto desempenho. Da mesma maneira, adicionamos material sobre modernos algoritmos distribuídos, entre eles algoritmos de sincronização de relógio e de localização baseados em GPS.

Embora isso não seja comum, conseguimos *reduzir* o número total de páginas. Essa redução é causada, em parte, pela exclusão de assuntos como coleta distribuída de lixo e protocolos de pagamento eletrônico, e também pela reorganização dos quatro últimos capítulos.

Como na edição anterior, o livro é dividido em duas partes. Princípios de sistemas distribuídos são discutidos do Capítulo 2 ao Capítulo 9, enquanto as abordagens globais dos modos de desenvolvimento de aplicações distribuídas (os paradigmas) são discutidas do Capítulo 10 ao Capítulo 13. Entretanto, diferente da edição anterior, decidimos não discutir estudos de casos completos nos capítulos dedicados a paradigmas. Em vez disso, agora cada princípio é explicado por meio de um caso representativo. Por exemplo, invocações de objeto são discutidas como um princípio de comunicação no Capítulo 10 sobre sistemas distribuídos baseados em objetos. Essa abordagem nos permitiu condensar o material, mas também tornou a leitura e o estudo mais agradáveis.

Claro que continuamos a recorrer extensivamente à prática para explicar o que são, afinal, sistemas distribuídos. Vários aspectos de sistemas utilizados na vida real, como WebSphere MQ, DNS, GPS, Apache, Corba, Ice, NFS, Akamai, TIB/Rendezvous, Jini e muitos outros são discutidos em todo o livro. Esses exemplos ilustram a tênue divisão entre teoria e prática, que torna a área de sistemas distribuídos tão interessante.

Muitas pessoas contribuíram para este livro de diversas maneiras. Gostaríamos de agradecer em especial a D. Robert Adams, Arno Bakker, Coskun Bayrak, Jacques Chassin de Kergommeaux, Randy Chow, Michel Chaudron, Puneet Singh Chawla, Fabio Costa, Cong Du, Dick Epema, Kevin Fenwick, Chandana Gamage, Ali Ghodsi, Giorgio Ingargiola, Mark Jelasity, Ahmed Kamel, Gregory Kapfhammer, Jeroen Ketema, Onno Kubbe, Patricia Lago, Steve MacDonald, Michael J. McCarthy, M. Tamer Ozsu, Guillaume Pierre, Avi Shahar, Swaminathan Sivasubramanian, Chintan Shah, Ruud Stegers, Paul Tymann, Craig E. Wills, Reuven Yagel e Dakai Zhu pela leitura de partes do manuscrito e por terem ajudado a identificar erros cometidos na edição anterior e oferecido comentários úteis.

Por fim, gostaríamos de agradecer a nossas famílias. Até agora, Suzanne já passou por esse processo dezessete vezes. É muito para mim, mas também para ela. Ela nunca disse: "Agora, chega!", embora eu tenha certeza de que teve vontade de dizer. Obrigado. Agora, Barbara e Marvin têm uma idéia muito melhor do que professores fazem para viver e sabem quais são as diferenças entre um bom e um mau livro didático. Agora eles são uma inspiração para eu tentar produzir mais bons do que maus livros (AST).

Como tirei uma licença para atualizar o livro, o trabalho de escrever também ficou muito mais agradável para Mariëlle. Ela está apenas começando a se acostumar com ele, mas continua a me dar suporte e a me alertar quando é

tempo de voltar minha atenção a assuntos mais importantes. Eu lhe devo muito. A essa altura, Max e Elke já têm uma idéia muito melhor do que significa escrever um livro, porém, em comparação com o que eles mesmos estão lendo, acham difícil entender o que há de tão interessante nessas coisas estranhas que chamamos sistemas distribuídos. E eu não posso culpá-los (MvS).

Companion Website



O Companion Website desta edição (www.prenhall.com/tanenbaum_br) oferece um conjunto completo de apresentações em PowerPoint para auxiliar os professores a preparar suas aulas, assim como manual de soluções em inglês. Esse material pode ser acessado por meio de uma senha, e, para obtê-la, os professores devem entrar em contato com um representante Pearson ou enviar um e-mail para universitarios@pearsoned.com.

Os editores da edição brasileira agradecem a preciosa colaboração do professor Fabio Kon.

1 Introdução

Os sistemas de computação estão passando por uma revolução. Desde 1945, quando começou a era moderna dos computadores, até aproximadamente 1985, os computadores eram grandes e caros. Mesmo os minicomputadores custavam no mínimo dezenas de milhares de dólares cada. O resultado é que a maioria das organizações tinha apenas alguns poucos computadores e, na falta de um modo de conectá-los, eles funcionavam independentemente uns dos outros.

Contudo, mais ou menos a partir de meados da década de 1980, dois avanços tecnológicos começaram a mudar essa situação. O primeiro foi o desenvolvimento de microprocessadores de grande capacidade. De início, eram máquinas de 8 bits, mas logo se tornaram comuns CPUs de 16, 32 e 64 bits. Muitas dessas CPUs tinham a capacidade de computação de um mainframe — isto é, um grande computador central —, mas por uma fração do preço dele.

A quantidade de melhorias que ocorreu na tecnologia de computadores nos últimos 50 anos é verdadeiramente assombrosa e totalmente sem precedentes em outros setores. De uma máquina que custava dez milhões de dólares e executava uma instrução por segundo, chegamos a máquinas que custam mil dólares e podem executar um bilhão de instruções por segundo, um ganho preço/desempenho de 10^{13} . Se os carros tivessem melhorado nessa proporção no mesmo período de tempo, um Rolls Royce custaria agora um dólar e faria um bilhão de milhas por galão. (Infelizmente, é provável que também tivesse um manual de 200 páginas para ensinar como abrir a porta.)

O segundo desenvolvimento foi a invenção de redes de computadores de alta velocidade. **Redes locais**, ou **LANs (local-area networks)**, permitem que centenas de máquinas localizadas dentro de um edifício sejam conectadas de modo tal que pequenas quantidades de informação possam ser transferidas entre máquinas em alguns microssegundos, ou algo parecido. Maiores quantidades de dados podem ser movimentadas entre máquinas a taxas de 100 milhões a 10 bilhões de bits/s. **Redes de longa distância**, ou **WANs (wide-area networks)**, permitem que milhões de máquinas no mundo inteiro se conectem a velocidades que variam de 64 Kbits/s a gigabits por segundo.

O resultado dessas tecnologias é que, atualmente, não somente é viável, mas também fácil, montar sistemas de computação compostos por grandes quantidades de computadores conectados por uma rede de alta velocidade. Esses sistemas costumam ser denominados redes de computadores ou **sistemas distribuídos**, em comparação com os **sistemas centralizados** (ou **sistemas monoprocessadores**) anteriores, que consistem em um único computador, seus periféricos e, talvez, alguns terminais remotos.

1.1 Definição de um Sistema Distribuído

Várias definições de sistemas distribuídos já foram dadas na literatura, nenhuma delas satisfatória e de acordo com nenhuma das outras. Para nossa finalidade, é suficiente dar uma caracterização sem ser muito específica:

Um sistema distribuído é um conjunto de computadores independentes que se apresenta a seus usuários como um sistema único e coerente.

Essa definição tem vários aspectos importantes. O primeiro é que um sistema distribuído consiste em componentes (isto é, computadores) autônomos. Um segundo aspecto é que os usuários, sejam pessoas ou programas, acham que estão tratando com um único sistema. Isso significa que, de um modo ou de outro, os componentes autônomos precisam colaborar. Como estabelecer essa colaboração é o cerne do desenvolvimento de sistemas distribuídos. Observe que nenhuma premissa é adotada em relação ao tipo de computadores. Em princípio, até mesmo dentro de um único sistema, eles poderiam variar desde computadores centrais (mainframes) de alto desempenho até pequenos nós em redes de sensores. Da mesma maneira, nenhuma premissa é adotada quanto ao modo como os computadores são interconectados. Voltaremos a esses aspectos mais adiante neste capítulo.

Em vez de continuar com definições, talvez seja mais útil que nos concentremos em características importantes de sistemas distribuídos. Uma característica importante é que as diferenças entre os vários computadores e o modo como eles se comunicam estão, em grande parte,

2 Sistemas distribuídos

ocultas aos usuários. O mesmo vale para a organização interna do sistema distribuído. Uma outra característica importante é que usuários e aplicações podem interagir com um sistema distribuído de maneira consistente e uniforme, independentemente de onde a interação ocorra.

Em princípio, também deveria ser relativamente fácil expandir ou aumentar a escala de sistemas distribuídos. Essa característica é uma consequência direta de ter computadores independentes, porém, ao mesmo tempo, de ocultar como esses computadores realmente fazem parte do sistema como um todo. Em geral, um sistema distribuído estará continuamente disponível, embora algumas partes possam estar temporariamente avariadas. Usuários e aplicações não devem perceber quais são as partes que estão sendo substituídas ou consertadas, ou quais são as novas partes adicionadas para atender a mais usuários ou aplicações.

Para suportar computadores e redes heterogêneos e, simultaneamente, oferecer uma visão de sistema único, os sistemas distribuídos costumam ser organizados por meio de uma camada de software — que é situada logicamente entre uma camada de nível mais alto, composta de usuários e aplicações, e uma camada subjacente, que consiste em sistemas operacionais e facilidades básicas de comunicação, como mostra a Figura 1.1. Por isso, tal sistema distribuído às vezes é denominado **middleware**.

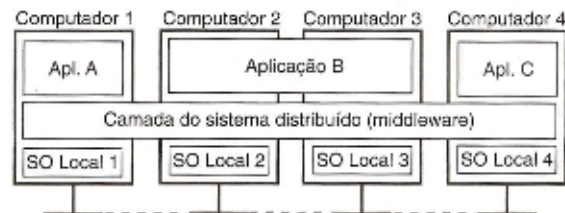


Figura 1.1 Sistema distribuído organizado como middleware. A camada de middleware se estende por várias máquinas e oferece a mesma interface a cada aplicação.

A Figura 1.1 mostra quatro computadores em rede e três aplicações, das quais a aplicação B é distribuída para os computadores 2 e 3. A mesma interface é oferecida a cada aplicação. O sistema distribuído proporciona os meios para que os componentes de uma única aplicação distribuída se comuniquem uns com os outros, mas também permite que diferentes aplicações se comuniquem. Ao mesmo tempo, ele oculta, do melhor e mais razoável modo possível, as diferenças em hardware e sistemas operacionais para cada aplicação.

1.2 Metas

O fato de ser possível montar sistemas distribuídos não quer dizer necessariamente que essa seja uma boa idéia. Afinal, dada a tecnologia corrente, também é possí-

vel colocar quatro drives de disco flexível em um computador pessoal. A questão é que não teria sentido fazer isso. Nesta seção, discutiremos quatro metas importantes que devem ser cumpridas na construção de um sistema distribuído para que valha a pena o esforço. Um sistema distribuído deve oferecer fácil acesso a seus recursos; deve ocultar razoavelmente bem o fato de que os recursos são distribuídos por uma rede; deve ser aberto e deve poder ser expandido.

1.2.1 Acesso a recursos

A principal meta de um sistema distribuído é facilitar aos usuários, e às aplicações, o acesso a recursos remotos e seu compartilhamento de maneira controlada e eficiente. Os recursos podem ser muito abrangentes, mas entre os exemplos típicos estão impressoras, computadores, facilidades de armazenamento, dados, páginas Web e redes, só para citar alguns. Há muitas razões para querer compartilhar recursos, e uma razão óbvia é a economia. Por exemplo, é mais barato permitir que uma impressora seja compartilhada por diversos usuários em um pequeno escritório do que ter de comprar e manter uma impressora direcionada a cada usuário. Do mesmo modo, em termos econômicos, faz sentido compartilhar recursos de alto custo como supercomputadores, sistemas de armazenamento de alto desempenho, imagesetters e outros periféricos caros.

Conectar usuários e recursos também facilita a colaboração e a troca de informações, o que é claramente ilustrado pelo sucesso da Internet com seus protocolos simples para trocar arquivos, correio, documentos, áudio e vídeo. Agora, a conectividade da Internet está levando a várias organizações virtuais nas quais grupos de pessoas muito dispersas geograficamente trabalham juntas por meio de **groupware**, isto é, software para edição colaborativa, teleconferência e assim por diante. Da mesma maneira, a conectividade da Internet possibilitou o comércio eletrônico, que nos permite comprar e vender todos os tipos de mercadoria sem ter de realmente ir a uma loja ou até mesmo sair de casa.

Contudo, à medida que a conectividade e o compartilhamento crescem, a segurança se torna cada vez mais importante. Na prática corrente, os sistemas oferecem pouca proteção contra bisbilhoteio ou intrusão na comunicação. Senhas e outras informações sensíveis muitas vezes são enviadas como texto comum — isto é, não criptografado — pela rede, ou armazenadas em servidores que podemos apenas esperar que sejam confiáveis. Nesse sentido, há muito espaço para melhorias. Por exemplo, hoje é possível fazer pedidos de mercadorias informando apenas um número de cartão de crédito. Raramente é solicitada uma prova de que o cliente seja o proprietário do cartão. No futuro, fazer pedidos desse modo só será possível se você realmente provar que possui o cartão em mãos, inserindo-o em uma leitora de cartões.

Um outro problema de segurança é o rastreamento de comunicações para montar um perfil de preferências de um usuário específico (Wang et al., 1998). Esse rastreamento é uma violação explícita da privacidade, em especial se for feito sem avisar o usuário. Um problema relacionado é que maior conectividade também pode resultar em comunicação indesejável, como envio de mala direta sem permissão, muitas vezes denominada *spam*. Nesses casos, talvez precisemos nos proteger usando filtros especiais de informações que selecionam mensagens de entrada com base em seu conteúdo.

1.2.2 Transparência da distribuição

Uma meta importante de um sistema distribuído é ocultar o fato de que seus processos e recursos estão fisicamente distribuídos por vários computadores. Um sistema distribuído que é capaz de se apresentar a usuários e aplicações como se fosse apenas um único sistema de computador é denominado **transparente**. Em primeiro lugar, vamos examinar quais são os tipos de transparência que existem em sistemas distribuídos. Em seguida, abordaremos a questão mais geral sobre a decisão de se a transparência é sempre requerida.

Tipos de transparência

O conceito de transparência pode ser aplicado a diversos aspectos de um sistema distribuído — os mais importantes são mostrados na Tabela 1.1.

Transparência	Descrição
Acesso	Oculta diferenças na representação de dados e no modo de acesso a um recurso
Localização	Oculta o lugar em que um recurso está localizado
Migração	Oculta que um recurso pode ser movido para outra localização
Relocação	Oculta que um recurso pode ser movido para uma outra localização enquanto em uso
Replicação	Oculta que um recurso é replicado
Concorrência	Oculta que um recurso pode ser compartilhado por diversos usuários concorrentes
Falha	Oculta a falha e a recuperação de um recurso

Tabela 1.1 Diferentes formas de transparência em um sistema distribuído (ISO, 1995).

Transparência de acesso trata de ocultar diferenças em representação de dados e o modo como os recursos podem ser acessados por usuários. Em um nível básico, desejamos ocultar diferenças entre arquiteturas de máquinas, porém o mais importante é chegar a um acordo sobre como os dados devem ser representados por máquinas e sistemas operacionais diferentes. Por exemplo, um sistema distribuído pode ter sistemas de computação que executam sistemas operacionais diferentes, cada um com suas próprias convenções para nomeação de arquivos.

Diferenças entre convenções de nomeação e também o modo como os arquivos devem ser manipulados devem ficar ocultos dos usuários e das aplicações.

Um importante grupo de tipos de transparência tem a ver com a localização de um recurso. **Transparência de localização** refere-se ao fato de que os usuários não podem dizer qual é a localização física de um recurso no sistema. A nomeação desempenha um papel importante para conseguir transparência de localização.

Em particular, pode-se conseguir transparência de localização ao se atribuir somente nomes lógicos aos recursos, isto é, nomes nos quais a localização de um recurso não está secretamente codificada. Um exemplo desse tipo de nome é o URL <http://www.prenhall.com/index.html>, que não dá nenhuma pista sobre a localização do principal servidor Web da Prentice Hall. O URL também não dá nenhuma pista se *index.html* sempre esteve em sua localização corrente ou se foi transferido para lá recentemente. Diz-se que sistemas distribuídos nos quais recursos podem ser movimentados sem afetar o modo como podem ser acessados proporcionam **transparência de migração**. Ainda mais vantajosa é a situação na qual recursos podem ser relocados *enquanto* estão sendo acessados sem que o usuário ou a aplicação percebam qualquer coisa. Nesses casos, diz-se que o sistema suporta **transparência de relocação**. Um exemplo de transparência de relocação é o uso móvel de laptops sem fio, cujos usuários podem continuar a usá-lo quando vão de um lugar a outro sem sequer se desconectar temporariamente.

Como veremos, a replicação desempenha um papel muito importante em sistemas distribuídos. Por exemplo, recursos podem ser replicados para aumentar a disponibilidade ou melhorar o desempenho colocando uma cópia perto do lugar em que ele é acessado. **Transparência de replicação** está relacionada a ocultar o fato de que existem várias cópias de um recurso. Para ocultar a replicação dos usuários, é necessário que todas as réplicas tenham o mesmo nome. Por consequência, um sistema que suporta transparência de replicação em geral também deve suportar transparência de localização porque, caso contrário, seria impossível referir-se a réplicas em diferentes localizações.

Já mencionamos que uma importante meta de sistemas distribuídos é permitir compartilhamento de recursos. Em muitos casos, esse compartilhamento é cooperativo, como no caso da comunicação. Todavia, também há muitos exemplos de compartilhamento competitivo de recursos. Um deles pode ser o caso de dois usuários independentes, em que cada um pode ter armazenado seus arquivos no mesmo servidor de arquivos ou pode acessar as mesmas tabelas em um banco de dados compartilhado. Nesses casos, é importante que cada usuário não perceba que o outro está utilizando o mesmo recurso. Esse fenômeno é denominado **transparência de concorrência**. Uma questão importante é que o acesso concorrente a um recurso compartilhado deixe esse recurso em estado con-

sistente. Pode-se conseguir consistência por meio de travas de acesso, o que dá a cada usuário, um por vez, acesso exclusivo ao recurso desejado. Um mecanismo mais refinado é utilizar transações; porém, como veremos em capítulos posteriores, é bastante difícil implementar transações em sistemas distribuídos.

Uma definição alternativa e popular de um sistema distribuído, devida a Leslie Lamport, é “Você sabe que tem um quando a falha de um computador do qual nunca ouviu falar impede que você faça qualquer trabalho”. Essa descrição pontua uma outra questão importante no projeto de sistemas distribuídos: como tratar falhas. Fazer com que um sistema distribuído seja **transparente à falha** significa que um usuário não percebe que um recurso (do qual possivelmente nunca ouviu falar) deixou de funcionar bem e que, subsequentemente, o sistema se recuperou da falha. Mascaram falhas é uma das questões mais difíceis em sistemas distribuídos e até mesmo impossíveis de se realizar quando são adotadas certas premissas aparentemente realistas, como discutiremos no Capítulo 8. A principal dificuldade para mascarar falhas está na incapacidade de distinguir entre um recurso morto e um recurso insuportavelmente lento. Por exemplo, quando contactamos um servidor Web ocupado, a certa altura o tempo do browser se esgotará e ele avisará que a página Web não está disponível. Nesse ponto, o usuário não pode concluir se, na verdade, o servidor está avariado.

Grau de transparência

Embora a transparência de distribuição seja geralmente considerada preferível para qualquer sistema distribuído, há situações em que tentar ocultar completamente dos usuários todos os aspectos da distribuição não é uma boa idéia. Um exemplo é requisitar que seu jornal eletrônico apareça em sua caixa postal antes das 7 horas da manhã, hora local, como sempre, enquanto naquele momento você está no outro lado do mundo, onde o fuso horário é diferente. Seu jornal matutino não será aquele ao qual você está acostumado.

Da mesma maneira, não se pode esperar que um sistema distribuído de longa distância que conecta um processo em San Francisco a um processo em Amsterdã oculte o fato de que a Mãe Natureza não permitirá que ele envie uma mensagem de um processo para o outro em menos do que aproximadamente 35 milissegundos. Na prática, quando se está usando uma rede de computadores, isso levará várias centenas de milissegundos. A transmissão de sinal não somente está limitada pela velocidade da luz, mas também pelas capacidades de processamento dos computadores intermediários.

Também há um compromisso entre um alto grau de transparência e o desempenho de um sistema. Por exemplo, muitas aplicações de Internet tentam contatar um servidor repetidas vezes antes de finalmente desistir. Por consequência, tentar mascarar uma falha transitória de

servidor antes de tentar um outro pode reduzir a velocidade do sistema como um todo. Nesse caso, talvez seja melhor desistir mais cedo ou, ao menos, permitir que o usuário cancele as tentativas para fazer contato.

Um outro exemplo é o de precisar garantir que várias réplicas, localizadas em continentes diferentes, devam ser consistentes o tempo todo. Em outras palavras, se uma cópia for alterada, essa alteração deve ser propagada para todas as cópias antes de permitir qualquer outra operação. É claro que, agora, uma única operação de atualização pode demorar até alguns segundos para ser concluída, algo que não pode ser ocultado dos usuários.

Por fim, há situações em que não é nem um pouco óbvio que ocultar distribuição seja uma boa idéia. À medida que sistemas distribuídos estão se expandindo para dispositivos que as pessoas carregam consigo, e a própria noção de localização e percepção de contexto está se tornando cada vez mais importante, pode ser melhor *expor* a distribuição em vez de tentar ocultá-la. Essa exposição de distribuição ficará mais evidente quando discutirmos sistemas distribuídos embutidos e ubíquos neste capítulo. Como um exemplo simples, considere uma funcionária que quer imprimir um arquivo que está em seu notebook. É melhor enviar o trabalho a ser impresso a uma impressora ocupada que está próxima do que a uma desocupada localizada na sede corporativa em um país diferente.

Também há outros argumentos contra a transparência de distribuição. Reconhecendo que a total transparência de distribuição é simplesmente impossível, devemos nos perguntar se é até mesmo sensato *dissimular* que podemos alcançá-la. Talvez seja muito melhor tornar a distribuição explícita, de modo que o usuário e o desenvolvedor de aplicação nunca sejam levados a acreditar que existe uma coisa denominada transparência. O resultado será que os usuários entenderão de maneira mais clara o comportamento, às vezes inesperado, de um sistema distribuído e, por isso, estarão mais bem preparados para lidar com esse comportamento.

A conclusão é que visar à transparência de distribuição pode ser uma bela meta no projeto e na implementação de sistemas distribuídos, mas deve ser considerada em conjunto com outras questões, como desempenho e facilidade de compreensão. Nem sempre vale a pena tentar conseguir total transparência, pois o preço que se paga por isso pode ser surpreendentemente alto.

1.2.3 Abertura

Uma outra meta importante de sistemas distribuídos é a abertura. Um **sistema distribuído aberto** é um sistema que oferece serviços de acordo com regras padronizadas que descrevem a sintaxe e a semântica desses serviços. Por exemplo, em redes de computadores há regras padronizadas que governam o formato, o conteúdo e o significado de mensagens enviadas e recebidas. Tais regras são formalizadas em protocolos. No caso de siste-